



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



DIGITAL SIGNAL PROCESSING

Debugging a Polyphase FIR Filter:Phase Indexing Error

Q) A polyphase FIR filter gives shifted output samples because the coefficient indexing begins at phase 1 instead of phase 0. Debug the implementation

Name: M.Megha Vardhan Reddy

Reg No: 192412324

Course: ECA0908 — Digital Signal Processing

Objective

A polyphase FIR filter shows shifted output samples. The cause: the coefficient phase index loop starts at $k = 1$ instead of $k = 0$. This deck identifies the bug, explains why it causes a 1-sample shift, fixes it, and verifies the fix with a MATLAB simulation.

1

Understand

Polyphase decomposition and correct phase indexing ($k = 0..M-1$)

2

Diagnose

Identify the off-by-one bug and trace its effect on $h[0]$

3

Fix & Verify

Correct the loop and confirm with a MATLAB simulation

Polyphase Decomposition — Background

$$h_k[n] = h[n \cdot M + k]$$
$$k = 0, 1, \dots, M-1$$

Each phase k picks every M -th coefficient starting at offset k . Branches run at the lower (decimated) rate and recombine to form the output.

Key rule: Phase 0 must contain $h[0]$ — the zero-delay tap.

Correct Phase Assignment ($M = 4$ example)

Phase k	Coefficients Picked
0	$h[0], h[4], h[8], \dots$
1	$h[1], h[5], h[9], \dots$
2	$h[2], h[6], h[10], \dots$
3	$h[3], h[7], h[11], \dots$

The Bug — Off-by-One Phase Index

```
for (k = 1; k <= M; k++) {    // WRONG: k starts at 1
    for (n = 0; n < N/M; n++) {
        h_poly[k][n] = h[n*M + k]; // shifts every phase by one tap
    }
}
```

Effect of the Bug

- Phase 1 (should hold $h[0]$, $h[4]$, $h[8]$...) instead holds $h[1]$, $h[5]$, $h[9]$...
- Phase 0 — meant to hold $h[0]$ — is never computed (index 0 unused)
- Every branch effectively uses the NEXT phase's coefficients

Consequence

Since $h[0]$ never reaches the zero-delay branch, the entire filter output is shifted by 1 sample. In multirate systems this also breaks commutator synchronization, showing up as a periodic phase jump.

Root Cause Analysis

Bloom Level 4 — Analyze

Symptom	Cause
Output shifted by 1 sample	Loop/array indexing off-by-one ($k = 1..M$ instead of $0..M-1$)
Phase 0 branch empty / garbage	Array index 0 never written to
Aliased / incorrect reconstruction in multirate use	Commutator expects branch order $0,1,...,M-1$; mismatch corrupts sync
Classic pattern: an off-by-one indexing error, common when translating 1-based mathematical notation (“phases 1 to M”) directly into 0-based array code without adjustment.	

Corrected Implementation

```
for (k = 0; k < M; k++) {           // FIX: k starts at 0
    for (n = 0; n < N/M; n++) {
        h_poly[k][n] = h[n*M + k]; // phase 0 gets h[0], h[M], h[2M]...
    }
}
```

Commutator / Output Combination

```
for (k = 0; k < M; k++) {
    y_poly[k] = 0;
    for (n = 0; n < N/M; n++)
        y_poly[k] += h_poly[k][n] * x_delay[k][n];
}
```

✓ Indexing phases from 0 to $M-1$ ensures phase 0 always contains $h[0]$, restoring correct time alignment.

MATLAB Simulation — Setup

Simulation compares three impulse responses: Direct-form FIR (reference), Corrected Polyphase ($k = 0..M - 1$), and Buggy Polyphase ($k = 1..M$).

```
N = 20; M = 4;           % filter length, phases
h = fir1(N-1, 0.25); h = h/sum(h); % low-pass FIR, normalized
x = [1, zeros(1,39)];    % impulse input

y_direct = conv(x, h);    % direct-form reference

% Correct: k = 0 : M-1
for k = 0:M-1
    for n = 0:Nsub-1
        h_poly_correct(k+1,n+1) = h(n*M + k + 1);

% Buggy: k = 1 : M  <-- BUG
for k = 1:M
    for n = 0:Nsub-1
        h_poly_buggy(k,n+1) = h(n*M + k + 1);
```


MATLAB Simulation — Result

Direct-form FIR

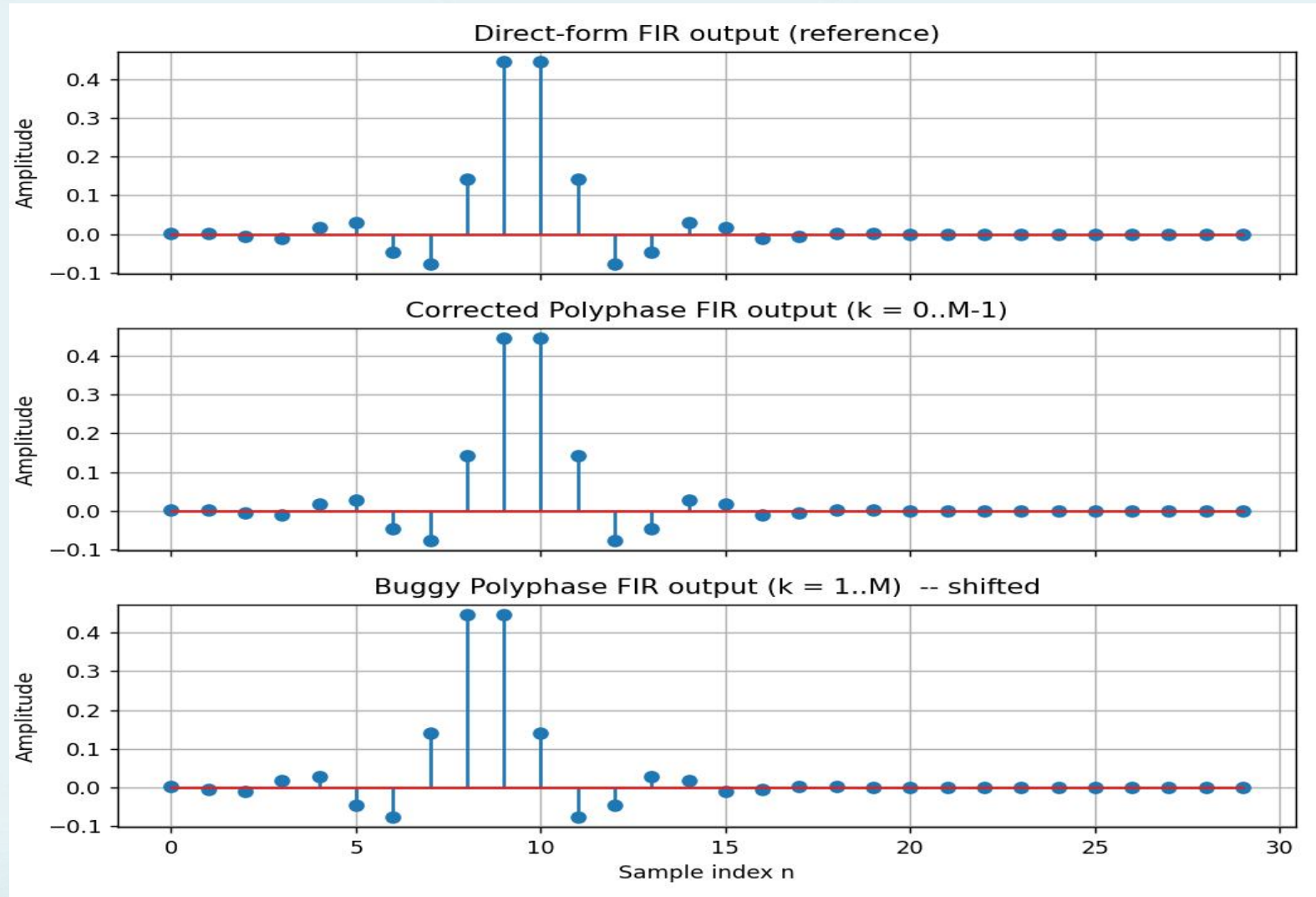
Peak at $n = 9$

Corrected Polyphase

Peak at $n = 9$ ✓

Buggy Polyphase

Peak at $n = 8$ (shift = 1)



Conclusion

The 1-sample output shift is caused by an off-by-one error in phase indexing (loop starting at $k = 1$ instead of $k = 0$), which excludes $h[0]$ from any branch and misaligns every coefficient-to-branch mapping.

- 1 Index phases from 0 to $M - 1$ so phase 0 always contains $h[0]$
- 2 Restores correct time alignment and group delay
- 3 Verified via MATLAB: peak aligns at $n = 9$ for both direct-form and corrected polyphase output

THANK YOU